部署实录

# 在 Docker 里的 TRSS-Yunzai 上跑扫码登录

**环境**Docker / TRSS_AllBot（Arch Linux，Node 26，Chromium 154）
**结论**微信零改动跑通，全链路含落库与战绩
**排障**图片的三个「对象陷阱」+ localhost URL 不可达

## 0. 一句话结论

**能跑，而且微信那条一行代码都不用改。** 容器里实测 937ms 出码，真机扫码后认出 角色「苦橘甘来 / 钻石III」。

QQ 那条也能跑，前提是容器里有浏览器 —— 你这台本来就有（Arch 的 `chromium` 154）。 我另外让插件**自己会去找系统浏览器**，将来换镜像、换机器也不用再下 100MB 的 Chromium。

真正需要动手的只有三件事：**同步代码、重启网关、装中文字体**。第三件跟扫码无关， 但跟你最终能不能看到图有关。

## 1. 勘察：你的容器实际是什么样

| 项目 | 实际值 | 影响 |
| --- | --- | --- |
| 容器名 / 镜像 | `TRSS_AllBot` / `trss:allbot` | — |
| 发行版 | Arch Linux | 包名跟 Debian 系不同，`chromium` 就是包名 |
| 云崽根目录 | `/root/TRSS_AllBot/TRSS-Yunzai` | 插件在 `plugins/Valorant-Plugin` |
| Node | **v26.11.1** | 比本地（22.x）新，实测无问题 |
| puppeteer | 25.13.0（在云崽根 `node_modules`） | 插件从上层 node_modules 解析到，不用单独装 |
| 插件自带依赖 | 只有 `yaml` | 登录模块零依赖，`git pull` 就能跑 |
| 浏览器 | `/usr/bin/chromium` → `Chromium 154.0.8037.97` | QQ 扫码要用 |
| puppeteer 浏览器缓存 | `~/.cache/puppeteer` **不存在** | 见 2.1，不影响 |
| 中文字体 | `fc-list :lang=zh` **为空** | 见 4.3，必须补 |
| `/dev/shm` | **64 MB** | 必须 `--disable-dev-shm-usage` |
| 内存 | 3.9 GB（可用 2.8 GB） | 够，浏览器空闲 5 分钟自动回收 |
| 网关端口 | 7820，只监听 `127.0.0.1` | 容器外访问要 `-p` |

## 2. 三个「看着像坑，其实不是」的地方

排查时最容易在这三处浪费半天，先说清楚。

### 2.1 puppeteer 装了但没下载浏览器 —— 不影响

`ls ~/.cache/puppeteer` 报「没有那个文件」，按常识 `puppeteer.launch()` 应该直接 报 `Could not find Chrome` 。但实测它是**成功**的，而且打印了一行 `[Chromium] /usr/sbin/chromium` 。

原因在云崽根目录那个 `.puppeteerrc.cjs` ：

```
// /root/TRSS_AllBot/TRSS-Yunzai/.puppeteerrc.cjs（镜像自带，不必改）
if (["linux", "android"].includes(process.platform))
  for (const item of ["chromium", "chromium-browser", "google-chrome", "google-chrome-stable"])
    try {
      const chromiumPath = execSync(`command -v ${item}`).toString().trim()
      if (chromiumPath && existsSync(chromiumPath)) { executablePath = chromiumPath; break }
    } catch {}
module.exports = { skipDownload: !!executablePath, executablePath }
```

TRSS 镜像烘焙时就写好了这套探测：**有系统浏览器就用系统的，跳过下载**。 所以「puppeteer 没浏览器」在这个镜像里是个假警报。

> 但这条兜底只对「在云崽根目录下启动的 node 进程」生效（puppeteer 从 cwd 往上找 `.puppeteerrc.cjs`）。插件做自测脚本、或者在别的目录起网关时，就吃不到这个配置 —— 这就是 3 里那层探测要自己写一遍的原因。

### 2.2 Chromium 154 比 puppeteer 25 预期的新 —— 没有影响

puppeteer 25 发布时配套的是 Chrome 13x。但本插件只用到 CDP 的通用能力：

- `newPage()` / `setUserAgent()` / `setViewport()`
- `evaluateOnNewDocument()`（抹 `navigator.webdriver`）
- `page.on('request')` / `page.on('framenavigated')`（抓 `auth://` 跳转）
- `page.$(selector)` + `elementHandle.screenshot()`

实测：启动 3.1s，开页、注入 UA、截图全部正常。**不用为了版本对齐去降级 Chrome。**

### 2.3 网关跑着老代码 —— 这个是真坑

改完文件不等于生效。网关是常驻进程，`login/` 下的适配器在**启动那一刻**就 `register` 进内存了，之后你 `docker cp` 覆盖文件，它照样用旧的。

症状很好认：插件发 `#瓦微信登录`，机器人回「网关没有挂载 微信 扫码适配器」， 可你明明是照文档配的。

```
# 判断方法：看启动横幅那行
grep '扫码适配器' /tmp/valorant-bridge.log | tail -1
# 旧代码：扫码适配器：qq
# 新代码：扫码适配器：qq / wx      ← 认这一行
```

顺手记一个反例：容器里还有一份 `config/config/config.yaml` 是**旧模板**（没有 `gateway.autoStart`）。这不冲突 —— 插件读配置时会拿 `config/default_config/config.yaml` 补缺的键，所以 `autoStart` 实际是 `true`。 但要知道这件事，否则「为什么我没开自动启动它自己起来了」会困惑很久。

---

## 3. 代码改动：让插件自己找浏览器

改动只在 `bridge/login/browser.js`，逻辑是「先找系统浏览器，找不到再退回 puppeteer 自带」。

```
import { existsSync } from 'node:fs'

const SYSTEM_CANDIDATES = [
  process.env.PUPPETEER_EXECUTABLE_PATH,
  process.env.CHROME_PATH,
  '/usr/bin/chromium', '/usr/bin/chromium-browser',
  '/usr/bin/google-chrome', '/usr/bin/google-chrome-stable',
  '/usr/sbin/chromium', '/opt/google/chrome/chrome', '/snap/bin/chromium',
  '/Applications/Google Chrome.app/Contents/MacOS/Google Chrome',
  '/Applications/Chromium.app/Contents/MacOS/Chromium',
  '/Applications/Microsoft Edge.app/Contents/MacOS/Microsoft Edge'
]

function detectExecutable () {
  for (const c of SYSTEM_CANDIDATES) {
    if (!c) continue
    try { if (existsSync(c)) return c } catch {}
  }
  return ''
}
```

优先级变成：

| 顺序 | 来源 | 命中条件 | 容器里 |
|---|---|---|---|
| 1 | `puppeteer-core` + 系统浏览器 | 探测到可执行文件，且装了 `puppeteer-core` | ×（没装这个包） |
| 2 | `puppeteer` + 系统浏览器 | 探测到可执行文件，有 `puppeteer` | ✅ 命中 |
| 3 | `puppeteer` 自带浏览器 | 没系统浏览器，但缓存里有 | — |
| 4 | 云崽宿主 `lib/puppeteer/puppeteer.js` | 上面都不行但宿主有 | 兜底 |
| 5 | 都没有 → 返回 null | 调用方降级到纯 HTTP | 不会走到 |

实测输出：

```
[login] 浏览器就绪（puppeteer(/usr/bin/chromium)）
source: puppeteer(/usr/bin/chromium) | 耗时 3104 ms
version: Chrome/154.8037.97
```

两个逃生阀：

| 环境变量 | 作用 |
|---|---|
| `PUPPETEER_EXECUTABLE_PATH=/opt/chrome/chrome` | 手动指定，最高优先 |
| `LOGIN_BROWSER=off` | 彻底禁用浏览器，QQ 强制降级纯 HTTP |
| `LOGIN_BROWSER_IDLE=600000` | 浏览器空闲回收时间（毫秒），默认 5 分钟 |

---

## 4. 部署步骤

### 4.1 同步文件

从本地仓库拷到容器。登录相关一共 11 个文件：

```
P=/root/TRSS_AllBot/TRSS-Yunzai/plugins/Valorant-Plugin
# 宿主机上执行（在仓库根目录）
for f in \
  bridge/login/wxqr.js bridge/login/qqpage.js bridge/login/browser.js \
  bridge/login/exchange.js bridge/login/qq.js bridge/login/wechat.js \
  bridge/login/session.js \
  apps/login.js apps/help.js package.json tools/test-login.mjs ; do
  docker cp "$f" "TRSS_AllBot:$P/$f"
done
```

> 前提：容器里的插件仓库比你本地那份**旧**。若反过来（容器里是 `git pull` 来的新版），覆盖前先 `git -C $P status` 看一眼，别把上游新提交盖掉。

不用 `pnpm install` —— 登录模块零依赖，`puppeteer` 由云崽根目录提供。

### 4.2 重启网关

```
# 容器内
pkill -f 'bridge/server.mjs'
cd /root/TRSS_AllBot/TRSS-Yunzai/plugins/Valorant-Plugin
PROVIDER=mateapp TOKEN=change-me PORT=7820 nohup node bridge/server.mjs >> /tmp/valorant-bridge.log 2>&1 &
tail -8 /tmp/valorant-bridge.log    # 确认「扫码适配器：qq / wx」
```

如果让插件 `autoStart` 自己拉，就跳过这一步（但只在网关**没在跑**时才会拉，所以旧进程得先杀掉）。

### 4.3 装中文字体（必须，跟扫码无关）

这是容器里最容易被忽略的一条。**没中文字体，战绩图全是豆腐块**：

```
# 装之前
fc-list :lang=zh        # 空
```

截图对比（同一段 HTML，`font-family: sans-serif`）：

| 装之前 | 装之后 |
| --- | --- |
|  |  |



```
# 装之后
pacman -S --needed --noconfirm noto-fonts-cjk   # 188 MiB 下载 / 299 MiB 装完
fc-match 'sans-serif:lang=zh'
# NotoSansCJK-Regular.ttc: "Noto Sans CJK KR" "Regular"
```

`fc-match` 报 KR 是正常的 —— Noto CJK 是一个 `.ttc`，简繁日韩共用字形，中文照样渲染正确（上面右图就是证据）。

**为什么这跟扫码有关**：扫码那张图是上游给的 JPEG，不经过 Chromium，所以出码一直正常；一旦扫码成功、要出「战绩图」，就变成 Chromium 截图 → 没字体就全是方块，看起来像"插件坏了"。

### 4.4 验证

```
# 容器内：只看适配器挂没挂上（不需要人扫）
curl -s -H 'Authorization: Bearer change-me' \
  http://127.0.0.1:7820/v1/login/adapters
# {"ok":true,"data":{"adapters":["qq","wx"]},...}

# 出码自测（微信，纯 HTTP，秒出）
cd /root/TRSS_AllBot/TRSS-Yunzai/plugins/Valorant-Plugin
SET_NO_OPEN=1 node tools/test-login.mjs wx

# 走网关全链路（出码 → 扫 → commit 落库 → 查战绩），和插件实际调用顺序一致
USER_ID=cli node tools/gw-login-check.mjs
```

出码图落在 `/tmp/login-wx.jpg` 或 `/tmp/gw-wx.jpg`，容器里没有桌面，要拿出来看：

```
# 宿主机上
docker cp TRSS_AllBot:/tmp/gw-wx.jpg ./qr.jpg && open qr.jpg      # macOS
docker cp TRSS_AllBot:/tmp/gw-wx.jpg ./qr.jpg && xdg-open qr.jpg  # Linux
```

---

## 5. 实测数据（你这台机器）

### 5.1 分环节耗时

| 环节 | 容器内 | 备注 |
|---|---|---|
| 加载登录模块 | < 300 ms | `import('./bridge/login/index.js')` → `adapters: qq/wx` |
| 微信出码（直连适配器） | **937 ms** | 46 KB JPEG，`appname` 回显「掌上无畏契约」 |
| 微信出码（走网关 HTTP） | **1.0–2.8 s** | 多一次 HTTP + 会话登记；波动来自上游取票 |
| 扫码 → 已扫 | 一次轮询内 | 2 s 间隔 |
| 已扫 → 确认 | **3 s** | `23:24:13` → `23:24:16` |
| 浏览器冷启动 | **3104 ms** | `puppeteer(/usr/bin/chromium)`，只发生一次 |
| QQ 出码 | **7258 ms** | 398×398 真二维码 |
| 二维码有效期 | 180 s | 上游控制，本地 `LOGIN_TTL` 只是计时 |

### 5.2 全链路结果（走网关，和插件调用顺序一致）

```
✅ 网关出码 2809ms  会话 f9db3309753c2ec5
   23:24:13 → scanned（已扫描，请在手机上点「确认登录」）
   23:24:16 → confirmed（登录成功：苦橘甘来）
🎉 profile: {"nickname":"苦橘甘来","roleName":"苦橘甘来","tierText":"钻石III"}
💾 落库: {"saved":true,"user":"cli","role":{"uid":"JA-10621b7…","name":"苦橘甘来","roleId":"1383005380427"}}
📋 账号：苦橘甘来（JA-10621b7…）
   最近 3 场：
   · 天枢云阙 | 竞技模式 | 炼狱 | 12-16-9 | ACS 221
   · 源工重镇 | 竞技模式 | 炼狱 | 9-8-4 | ACS 304
   · 亚海悬城 | 竞技模式 | 芮娜 | 12-12-5 | ACS 296
   汇总: {"matches":3,"wins":1,"losses":1,"winRate":0.5,"acs":274,"kd":1.45}
```

落库文件 `bridge/accounts/cli.json`：

| 项 | 值 |
| --- | --- |
| 顶层字段 | `login` / `role` / `updatedAt` / `source` |
| cookie | **8 段**：clientType / appid / acctype / openid / access_token / userId / accountType / tid |
| scene | **175 字符**，`v3_` 开头 |
| source | `scan:wx`（扫码来的；抓包导入的仍是 `import`） |
| 文件权限 | `600`（`-rw-------`，只有 root 能读） |

> **`source` 那个字段是这次顺手修的**：`saveAccount()` 的默认值是 `'import'`，扫码成功落库也会被标成「抓包导入」，排查时容易看错凭证来源。现在 `saveLogin()` 会把 `type` 一起传下去，写成 `scan:wx` / `scan:qq`。

**这一步的意义**大于"登录成功"：它同时证明 —— 容器里的网关能出码、能轮询、能换票、能把凭证按人落库、能拿这份凭证查到真实战绩。也就是插件里 `#瓦微信登录` 那条路，除了「出图发给 QQ 群」这一段（要等云崽起来），其余全跑通了。

---

## 6. 容器专属的坑（清单）

| 现象 | 原因 | 处理 |
| --- | --- | --- |
| Chromium 起不来，报 sandbox 相关 | 容器以 root 跑 | 启动参数已含 `--no-sandbox --disable-setuid-sandbox` |
| 打开页面卡死 / 浏览器莫名崩溃 | `/dev/shm` 只有 64 MB | 已含 `--disable-dev-shm-usage`；或 `docker run --shm-size=1g` |
| 出图全是方块 | 没 CJK 字体 | 见 4.3 |
| 「网关没有挂载 微信 扫码适配器」 | 网关跑着老代码 | 重启网关，看横幅 |
| 端口占不住 / 起来就退 | 旧的 `server.mjs` 还在 | `pkill -f bridge/server.mjs` 再起 |
| 容器外连不上 7820 | 只监听 `127.0.0.1` | `docker run -p 127.0.0.1:7820:7820`，建议仍绑回环 |
| 重建容器后登录态没了 | `bridge/accounts/` 在容器内 | 挂卷：`-v valorant-accounts:/root/TRSS_AllBot/TRSS-Yunzai/plugins/Valorant-Plugin/br` |
| 日志里「续期成功 1 份，失败 3 份」 | 旧凭证的票据已过期 | 正常，重新扫一次即可 |
| 「登录态失效（result=999999）」 | 同上 | 同上 |
| **QQ 里气泡显示「消息加载失败」** | 图片被当成文本发了，见 6.1 | 用 `imageSegment()` 包一下 |
| 「图片分辨率检测错误 `unsupported file type`」 | `file` 传成了对象（`[object Object]`），见 6.2 | `file` 必须是 `base64://…` 字符串 |
| `ENOENT: open '[object Object]'`（在 `uploadMedia` 里） | 把 `screenshot()` 返回的**段**当成了 Buffer，见 6.2 | 递归解包 `.file` 到底 |
| 图发出去了但 QQ 里是空白 | 图片 URL 是 `localhost`，QQ 访问不到，见 6.3 | `QQBot.yaml` 里 `markdown: {QQ号: legacy}` |

### 6.1 「消息加载失败」到底是怎么回事（值得单独说）

这个现象跟扫码、跟容器都没关系，但最容易在容器里撞上——因为 TRSS_AllBot 默认接的是 **QQ 官方机器人**（`@windtrace/qq-group-bot`），而不是 icqq / OneBot。

日志里是这么写的：

```
[23:29:57][INFO][3889734773] send to User(AEE5AF04...): <markdown,content={"0":255,"1":216,"2":255,"3":224,...
                                                                                ↑ FF D8 FF E0 = JPEG 文件头
```

**整张二维码被 JSON 化塞进了 markdown 正文。** 客户端当然渲染不出来，就显示「消息加载失败」。

根因在 `plugins/QQBot-Plugin/adapter/build.js`：

```
if (typeof i === "object") i = { ...i }
else i = { type: "text", text: Bot.String(i) }
```

- 插件这边 `e.reply(buffer)` 传的是 **Buffer**；
- Buffer 是对象 → `{...buffer}` 变成 `{0:255, 1:216, 2:255, 3:224, …}`；
- 展开后没有 `type` 字段 → 进 `default:` 分支 → `makeRawMarkdownText(Bot.String(i))` → 变成文本；
- 于是 `content` 就是字节数组的 JSON。

`icqq` / `OneBot` 接受裸 Buffer，所以这个坑**只在官方机器人协议端出现**——换框架看起来好好的，一上 QQBot 就炸。

修法（`utils/render.js`）：把二进制统一归一到 Buffer，再拼成**字符串形式**的 `file`。

```
export function imageSegment (img) {
  const buf = toBuffer(img)                // Buffer / Uint8Array / ArrayBuffer 都收
  if (buf) return { type: 'image', file: `base64://${buf.toString('base64')}` }
  const file = String(img)
  if (typeof globalThis.segment?.image === 'function') {
    try { return globalThis.segment.image(file) } catch {}
  }
  return { type: 'image', file }
}
```

两个必须踩对的点：

1. **`file` 必须是字符串**，不能是对象。QQBot 取图片数据是 `Bot.Buffer(_file.file)`，而 `Bot.Buffer()` 只对字符串做解析（`base64://` / `http://` / 文件路径），拿到对象会先 `String()` 成 `"[object Object]"`。
2. **不能只判 `Buffer.isBuffer`**。puppeteer 25 的 `page.screenshot()` 返回的是 `Uint8Array`（实测 `Buffer.isBuffer(shot) === false`），漏判就会走 `String(uint8array)`，变成 `"255,216,255,224,…"` 这种字符串，照样不是图片。

> 顺带：`markdown` 元素（`msg_type=2`）在官方机器人上需要申请模板 / 权限。本插件回复的都是纯文本 + 图片，不涉及模板消息；如果你自己加了 markdown 卡片而显示空白，先去开放平台确认权限，别怀疑代码。

### 6.2 「图片分辨率检测错误」—— 和 6.1 是同一个病根的第二层

修完 6.1 之后日志里会冒出新的报错：

```
[ERRO] 图片分辨率检测错误 <Buffer 5b 6f 62 6a 65 63 74 20 4f 62 6a 65 63 74 5d>
TypeError: unsupported file type: undefined
    at imageSize (…/image-size@2.0.4/…/lookup.js:27:11)
    at QQBotAdapter.makeMarkdownImage (.../QQBot-Plugin/adapter/build.js:107:26)
```

把那个 Buffer 解出来是 `[object Object]`（`5b 6f 62 6a 65 63 74 …` 就是它的 ASCII）。也就是说 `Bot.Buffer()` 收到的不是图片数据，而是一个对象的字符串形式。

对照 `build.js` 和 `lib/util.js` 就很清楚了：

```
// build.js:106
const file = _file.url || _file.file
const buffer = await Bot.Buffer(file)

// lib/util.js:294
async Buffer(data) {
  if (!Buffer.isBuffer(data)) {
    data = this.String(data)            // ← 对象 → "[object Object]"
    if (data.startsWith("base64://")) …
  }
}
```

所以 6.1 的修法里，**`file` 一定要是字符串**。用 `segment.image(buffer)` 那种「file 是对象」的段，就会精确复现这个报错。

**还有一层没说完**：修完 `file` 是字符串之后，如果直接 `imageSegment(段)`， 报错会换个样子 —— 从「分辨率检测错误」变成：

```
发送消息错误 [ { type: 'image', file: '[object Object]', name: undefined } ]
ENOENT: no such file or directory, open '[object Object]'
    at getBase64FromLocal (qq-group-bot/lib/utils.js:33:22)
    at Bot.uploadMedia    (qq-group-bot/lib/qqBot.js:106:53)
```

因为云崽的 `screenshot()` **返回的不是 Buffer，而是已经包好的段**：

```
// lib/puppeteer/puppeteer.js
renderer.screenshot = async (name, data) => {
  const img = await renderer.render(name, data)   // Buffer
  return img ? segment.image(img) : img           // → { type:'image', file: Buffer }
}
```

所以插件里 `await puppeteer.screenshot(...)` 拿到的是 `{type:'image', file: Buffer}`。把它整个丢进 `imageSegment()` 再 `String()` 一次，又是 `"[object Object]"`。

正解是**递归解包**，不管套几层都拆到底：

```
function fileString (input, depth = 0) {
  if (input == null || depth > 4) return ''
  const buf = toBuffer(input)                      // Buffer / TypedArray / ArrayBuffer
  if (buf) return buf.length ? `base64://${buf.toString('base64')}` : ''
  if (typeof input === 'string') return input
  if (typeof input === 'object') {                 // 段: {type:'image', file|data}
    const next = input.file ?? input.data ?? input.buffer
    if (next != null && next !== input) return fileString(next, depth + 1)
  }
  return ''
}
```

实测四种输入都能拿到同一个 `base64://…`：

| 输入 | 解析结果 |
| --- | --- |
| `Buffer` | `base64:///9j/4BEi…` |
| `Uint8Array`（puppeteer 25 的返回值） | `base64:///9j/4BEi…` |
| `{type:'image', file: Buffer}`（云崽 screenshot） | `base64:///9j/4BEi…` |
| `{type:'image', file: {type:'image', file: Buffer}}`（双层） | `base64:///9j/4BEi…` |

> 一句话记法：**这条链上有三个「对象陷阱」** —— 元素是对象（Buffer 被展开）、`file` 是对象（`segment.image` 的产出）、返回值是对象（`screenshot` 的产出）。全都归一成字符串就没事了。

### 6.3 图片还有第三关：`localhost` 的 URL，QQ 服务器访问不到

**这是最容易漏的一条。** 分辨率修好后，图片能"发出去"，但 QQ 里可能还是空的。

原因：QQBot 默认走 **markdown 模式**，图片会被转成 markdown 图片语法：

```
![图片 #800px #600px](http://localhost:2536/File/xxxxxxxx)
```

那个 URL 来自 `config/config/server.yaml` 的 `url: http://localhost:2536`， 由云崽自己的 HTTP 服务提供。**QQ 的服务器不可能访问到你的 localhost。**

适配器本来有个 `toBotUpload: true` 的兜底（借一个 icqq 账号的 `uploadImage` 把图传到 QQ 图床），但这个容器里只有 QQBot 账号，没有 icqq —— `_uploadImageBot` 找不到， 于是退回本地 URL。

**解**：让 QQBot 走「原生消息元素」，图片就会用 `uploadMedia()` 富媒体上传 （`msg_type=7` + `file_info`），完全不依赖公网 URL。改 `config/QQBot.yaml`：

```
markdown:
  "3889734773": legacy      # 你的机器人 QQ 号
  "4010481450": legacy
```

`legacy` 这个值是 `adapter/send.js` 认的开关：

```
if (config.markdown[data.self_id] === "legacy") msgs = await this.makeMsg(data, msg)
else msgs = await this.makeRawMarkdownMsg(data, msg)   // ← 默认，图片变 markdown URL
```

改完重启云崽。代价是没有 markdown 排版（本来也用不上），换来图片能真正显示。

> 如果你**有**公网域名，也可以不改 legacy，改把 `server.yaml` 的 `url` 指到公网地址 （记得放行 2536 端口）。两条路选一条，混着用会互相打架。

---

## 7. 命令速查

```
# ---- 宿主机 ----
docker ps --format '{{.Names}}\t{{.Status}}'                    # 容器活着吗
docker exec TRSS_AllBot bash -lc 'tail -20 /tmp/valorant-bridge.log'
docker cp TRSS_AllBot:/tmp/gw-wx.jpg ./qr.jpg                   # 取二维码

# ---- 容器内 ----
P=/root/TRSS_AllBot/TRSS-Yunzai/plugins/Valorant-Plugin
curl -s -H 'Authorization: Bearer change-me' http://127.0.0.1:7820/health
curl -s -H 'Authorization: Bearer change-me' http://127.0.0.1:7820/v1/login/adapters
ps aux | grep server.mjs | grep -v grep
cd $P && SET_NO_OPEN=1 node tools/test-login.mjs wx
cd $P && USER_ID=cli node tools/gw-login-check.mjs
```

---

## 8. 排障顺序（别乱试）

1. `curl /health` 通不通 → 不通说明网关没起 / 端口冲突
2. `curl /v1/login/adapters` 有没有 `wx` → 没有说明网关是老代码，重启
3. `node tools/test-login.mjs wx` 出不出码 → 不出说明上游或网络问题（与容器无关）
4. 出码但扫不动 → 码过期（180s），重发；或微信侧风控
5. 扫成功但落库失败 → 看 commit 返回的 `hint`；多为 `scene` 缺失，去 App 里进一次「我的 → 战绩」
6. 都能跑但图是方块 → 字体（4.3）
7. **QQ 里显示「消息加载失败」** → 先翻日志找 `<markdown,content={"0":255,…}>`， 命中就是 Buffer 被当文本发了（6.1）。注意这一步要在**包成 image 段之后**再测， 否则容器里怎么调都正常、一到 QQ 就炸 —— 因为问题出在协议端，不在容器。

> 最后这条的排查经验值得记住：**「容器里调通了」和「QQ 里能看见」是两件事**。 前者验证的是业务逻辑（出码、换票、落库、取数），后者验证的是消息序列化。 两条链任何一条断，用户看到的都是「没反应」或「加载失败」。